

**LAPORAN PRAKTIKUM SISTEM OPERASI
MANAJEMEN PROSES DAN CHAT IPC**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

SISTEM MANAJEMEN PROSES

Sistem Manajemen Proses adalah bagian dari sistem operasi yang bertanggung jawab untuk mengatur dan mengelola semua proses yang berjalan di dalam komputer.

Proses sendiri adalah program yang sedang dieksekusi beserta sumber daya yang digunakannya (CPU, memori, I/O).

Tujuan

- Memahami cara kerja fork() untuk membuat proses anak pada sistem operasi.
- Mengimplementasikan tiga proses anak dengan tugas berbeda (sorting, searching, perhitungan prima).
- Memahami komunikasi antar proses (IPC) menggunakan pipe.
- Menampilkan informasi proses: PID, waktu eksekusi, status keluar, dan hasil tugas.

a. Proses

Proses adalah program yang sedang dieksekusi oleh CPU, yang terdiri atas kode program, data, stack, dan informasi eksekusi.

b. Manajemen Proses

Manajemen proses adalah layanan sistem operasi untuk:

- Membuat dan menghentikan proses.
- Menjadwalkan proses agar CPU digunakan efisien.
- Mengatur komunikasi antar proses (IPC).
- Menyediakan sinkronisasi agar tidak terjadi konflik.

Status proses:

1. New → proses baru dibuat.
2. Ready → siap dijalankan.
3. Running → sedang dijalankan oleh CPU.
4. Waiting → menunggu event tertentu.
5. Terminated → proses selesai.

IPC (Inter-Process Communication)

IPC adalah mekanisme komunikasi antar proses. Pada percobaan ini digunakan pipe, yaitu media komunikasi satu arah antara parent dan child process.

Alat dan Bahan

- Sistem Operasi: Linux (Ubuntu)
- Compiler: GCC
- Bahasa Pemrograman: C

Input

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>
```

```
typedef struct {
    pid_t pid;
    int task_id;
    double waktu_ms;
    int kode_keluar;
    int64_t hasil;
} hasil_proses_t;
```

```
double selisih_waktu_ms(const struct timespec *mulai, const struct timespec *selesai) {
    double s = (double)(selesai->tv_sec - mulai->tv_sec) * 1000.0;
    double ns = (double)(selesai->tv_nsec - mulai->tv_nsec) / 1e6;
    return s + ns;
}
```

```
int cmp(const void *a, const void *b) {
    int ia = *(const int*)a;
    int ib = *(const int*)b;
    return (ia > ib) - (ia < ib);
}
```

```
int64_t tugas_sorting(size_t n) {
    int *arr = malloc(n * sizeof(int));
    if (!arr) return -1;
    for (size_t i = 0; i < n; ++i) arr[i] = rand();
    qsort(arr, n, sizeof(int), cmp);
    int64_t jumlah = 0;
```

```

    for (size_t i = 0; i < n; ++i) jumlah += arr[i];
    free(arr);
    return jumlah;
}

```

```

int64_t tugas_searching(size_t n, int target) {
    int *arr = malloc(n * sizeof(int));
    if (!arr) return -2;
    for (size_t i = 0; i < n; ++i) arr[i] = (int)i;
    ssize_t kiri = 0, kanan = (ssize_t)n - 1;
    while (kiri <= kanan) {
        ssize_t tengah = (kiri + kanan) / 2;
        if (arr[tengah] == target) {
            free(arr);
            return tengah;
        } else if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    free(arr);
    return -1;
}

```

```

int64_t tugas_hitung_prima(int N) {
    if (N < 2) return 0;
    char *prima = malloc((N+1) * sizeof(char));
    if (!prima) return -1;
    memset(prima, 1, N+1);
    prima[0]=prima[1]=0;
    for (int p=2; p*p<=N; ++p) {
        if (prima[p]) {
            for (int q=p*p; q<=N; q+=p) prima[q]=0;
        }
    }
    int64_t hitung=0;
    for (int i=2; i<=N; ++i) if (prima[i]) ++hitung;
    free(prima);
    return hitung;
}

```

```

int main(void) {
    srand((unsigned int)time(NULL) ^ (unsigned int)getpid());

    const int JUMLAH_TUGAS = 3;
    int pipes[JUMLAH_TUGAS][2];
    pid_t pid_anak[JUMLAH_TUGAS];

    for (int i = 0; i < JUMLAH_TUGAS; ++i) {
        if (pipe(pipes[i]) == -1) {
            perror("gagal membuat pipe");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

pid_t pid = fork();
if (pid < 0) {
    perror("gagal fork");
    exit(EXIT_FAILURE);
} else if (pid == 0) {
    close(pipes[i][0]);

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    hasil_proses_t hasil;
    memset(&hasil, 0, sizeof(hasil));
    hasil.pid = getpid();
    hasil.task_id = i+1;
    hasil.kode_keluar = 0;
    hasil.hasil = 0;

    if (i == 0) {
        size_t n = 100000;
        hasil.hasil = tugas_sorting(n);
    } else if (i == 1) {
        size_t n = 100000;
        int target = rand() % (int)n;
        hasil.hasil = tugas_searching(n, target);
    } else if (i == 2) {
        int N = 200000;
        hasil.hasil = tugas_hitung_prima(N);
    }

    clock_gettime(CLOCK_MONOTONIC, &t1);
    hasil.waktu_ms = selisih_waktu_ms(&t0, &t1);

    write(pipes[i][1], &hasil, sizeof(hasil));
    close(pipes[i][1]);
    exit(0);
} else {
    pid_anak[i] = pid;
    close(pipes[i][1]);
}
}

printf("PID Induk: %d, menunggu %d proses anak...\n\n", getpid(), JUMLAH_TUGAS);
for (int i = 0; i < JUMLAH_TUGAS; ++i) {
    int status;
    pid_t w = waitpid(pid_anak[i], &status, 0);
    if (w == -1) {
        perror("gagal waitpid");
        continue;
    }

    hasil_proses_t hasil;
    ssize_t r = read(pipes[i][0], &hasil, sizeof(hasil));

```

```

if (r <= 0) {
    hasil.pid = pid_anak[i];
    hasil.task_id = i+1;
    hasil.waktu_ms = 0.0;
    hasil.kode_keluar = WIFEXITED(status) ? WEXITSTATUS(status) : -1;
    hasil.hasil = 0;
}

char status_str[128];
if (WIFEXITED(status)) {
    snprintf(status_str, sizeof(status_str), "SELESAI (kode keluar=%d)",
WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    snprintf(status_str, sizeof(status_str), "TERBUNUH oleh sinyal %d", WTERMSIG(status));
} else {
    snprintf(status_str, sizeof(status_str), "TIDAK DIKETAHUI");
}

printf("PID Anak : %d\n", hasil.pid);
printf(" Tugas      : %d (%s)\n", hasil.task_id,
    hasil.task_id==1 ? "Pengurutan (Sorting)" :
    (hasil.task_id==2 ? "Pencarian (Searching)" : "Perhitungan Prima"));
printf(" Waktu Eksekusi: %.3f ms\n", hasil.waktu_ms);
printf(" Status Keluar : %s\n", status_str);
if (hasil.task_id == 1) {
    printf(" Hasil (jumlah elemen terurut) : %lld\n", (long long)hasil.hasil);
} else if (hasil.task_id == 2) {
    if (hasil.hasil >= 0)
        printf(" Hasil (indeks ditemukan)      : %lld\n", (long long)hasil.hasil);
    else
        printf(" Hasil (pencarian)                : tidak ditemukan\n");
} else if (hasil.task_id == 3) {
    printf(" Hasil (jumlah bilangan prima) : %lld\n", (long long)hasil.hasil);
}
printf("\n");

close(pipes[i][0]);
}

printf("Semua proses anak selesai. Proses induk keluar.\n");
return 0;
}

```

Desain Program

1. Parent process membuat 3 child process.
2. Tiap child process menjalankan tugas berbeda:
 - Child 1: Sorting array → hasil berupa jumlah elemen.

- Child 2: Searching nilai dalam array terurut.
 - Child 3: Menghitung jumlah bilangan prima.
3. Hasil tiap child dikirim ke parent lewat pipe.
 4. Parent menunggu semua child selesai, lalu menampilkan ringkasan eksekusi.

Output

```
yos@yos-QEMU-Virtual-Machine:~$ ./sistem_manajemen_proses
PID Induk: 4048, menunggu 3 proses anak...

PID Anak : 4049
Tugas      : 1 (Pengurutan (Sorting))
Waktu Eksekusi: 23.939 ms
Status Keluar : SELESAI (kode keluar=0)
Hasil (jumlah elemen terurut) : 107422553752956

PID Anak : 4050
Tugas      : 2 (Pencarian (Searching))
Waktu Eksekusi: 0.332 ms
Status Keluar : SELESAI (kode keluar=0)
Hasil (indeks ditemukan) : 73108

PID Anak : 4051
Tugas      : 3 (Perhitungan Prima)
Waktu Eksekusi: 0.024 ms
Status Keluar : SELESAI (kode keluar=0)
Hasil (jumlah bilangan prima) : 17984

Semua proses anak selesai. Proses induk keluar.
yos@yos-QEMU-Virtual-Machine:~$
```

Pembahasan:

- Program berhasil membuat 3 proses anak dengan `fork()`.
- Setiap child menjalankan tugas berbeda sesuai perancangan.
- Parent menggunakan `waitpid()` untuk menunggu setiap anak selesai.
- Data hasil dieksekusi berhasil dikirim dari child ke parent melalui **pipe**.
- Waktu eksekusi berbeda tergantung kompleksitas tugas:
 - Sorting lebih cepat dibanding menghitung bilangan prima.
 - Searching sangat cepat karena hanya melakukan binary search.

Kesimpulan

1. **Manajemen proses** dalam sistem operasi memungkinkan eksekusi program secara paralel menggunakan banyak proses.
2. Dengan `fork()` parent dapat membuat child process yang independen.
3. Komunikasi antar proses berhasil dilakukan menggunakan **pipe**.
4. Informasi penting (PID, waktu eksekusi, status, hasil) dapat ditampilkan sehingga memudahkan pemantauan proses.

CHAT IPC

Chat IPC (Inter-Process Communication) adalah sebuah aplikasi chat sederhana yang dibuat dengan memanfaatkan mekanisme komunikasi antar proses pada sistem operasi. Dalam Chat IPC, setiap pengguna dijalankan sebagai proses yang terpisah, lalu pesan yang diketik oleh satu pengguna akan dikirim ke proses lain menggunakan sarana IPC seperti Message Queue, Shared Memory, atau Pipe. Dengan cara ini, beberapa proses dapat saling bertukar pesan seolah-olah mereka sedang melakukan percakapan (chat).

Tujuan

1. Mempelajari konsep **Inter-Process Communication (IPC)**.
2. Mengimplementasikan komunikasi antar proses menggunakan **Message Queue**.
3. Membuat aplikasi **chat sederhana** yang dapat digunakan oleh beberapa proses secara bersamaan.
4. Menggunakan *fork()* untuk membuat proses pengirim dan penerima.

Dasar Teori

a. Inter-Process Communication (IPC)

IPC adalah mekanisme yang memungkinkan proses-proses dalam sistem operasi untuk saling bertukar data dan berkomunikasi. Dalam sistem multiprogramming, proses sering kali perlu bekerja sama, sehingga IPC menjadi sangat penting.

Beberapa metode IPC di UNIX/Linux antara lain:

- **Pipe**: komunikasi satu arah antar proses yang memiliki hubungan parent-child.
- **Message Queue**: komunikasi berbasis antrian pesan, lebih fleksibel, dan dapat digunakan antar proses yang tidak berhubungan langsung.
- **Shared Memory**: memori bersama yang dapat diakses oleh banyak proses.
- **Semaphore/Mutex**: digunakan untuk sinkronisasi agar tidak terjadi race condition.

b. Message Queue

Message Queue merupakan mekanisme IPC yang menggunakan **antrian** untuk menyimpan pesan. Proses pengirim (*sender*) menaruh pesan ke antrian, sedangkan proses penerima (*receiver*) mengambil pesan dari antrian.

Keuntungan:

- Bisa menyimpan banyak pesan.

- Mendukung komunikasi antar proses yang tidak memiliki hubungan langsung.
- Pesan memiliki tipe, sehingga proses bisa mengambil pesan tertentu.

c. Sistem Chat dengan IPC

Pada program ini, dibuat sebuah aplikasi **chat sederhana** menggunakan message queue. Setiap user menjalankan program yang sama, tetapi diberi ID berbeda (User1, User2, dst). Program menggunakan `fork()` untuk membuat dua proses:

- **Proses anak** → menerima pesan dari message queue.
- **Proses induk** → mengirim pesan ke message queue.

Alat dan Bahan

- Sistem operasi Linux (Ubuntu, Debian, Fedora, dll).
- Compiler **gcc**.
- Library standar C (`stdio.h`, `stdlib.h`, `string.h`, `sys/ipc.h`, `sys/msg.h`, `unistd.h`).

Input

```
#define _POSIX_C_SOURCE 200809L
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#include <fcntl.h>
```

```
#include <sys/mman.h>
```

```
#include <sys/stat.h>
```

```
#include <semaphore.h>
```

```
#include <sys/types.h>
```

```
#include <errno.h>
```

```
#define SHM_NAME "/chat_shm_id"

#define SEM_NAME "/chat_mutex_id"
```

```
#define MAX_USER 32

#define MAX_TEXT 256

#define MAX_MSG 100
```

```
typedef struct {

    pid_t pid;

    char user[MAX_USER];

    char text[MAX_TEXT];

    time_t ts;

} chat_msg_t;
```

```
typedef struct {

    int clients;

    int head;

    int count;

    chat_msg_t msgs[MAX_MSG];

} chat_shm_t;
```

```
static chat_shm_t *shm = NULL;

static sem_t *mutex = NULL;

static int shm_fd = -1;
```

```
static void cleanup_on_exit(void) {

    if (!shm || !mutex) return;
```

```

sem_wait(mutex);

if (shm->clients > 0) shm->clients--;

int sisa = shm->clients;

sem_post(mutex);

if (sisa == 0) {

    sem_close(mutex);

    sem_unlink(SEM_NAME);

    munmap(shm, sizeof(chat_shm_t));

    close(shm_fd);

    shm_unlink(SHM_NAME);

} else {

    sem_close(mutex);

    munmap(shm, sizeof(chat_shm_t));

    close(shm_fd);

}

}

```

```

static void tampilkan_riwayat(void) {

    sem_wait(mutex);

    int c = shm->count;

    int h = shm->head;

    if (c == 0) {

        printf("[Belum ada pesan]\n");

        sem_post(mutex);

        return;

    }

    printf("=== CHAT ROOM ===\n");

    for (int i = 0; i < c; ++i) {

```

```

    int idx = (h + i) % MAX_MSG;

    chat_msg_t *m = &shm->msgs[idx];

    printf("%s> %s\n", m->user, m->text);

}

sem_post(mutex);
}

```

```

static void kirim_pesan(const char *user, const char *teks) {

    chat_msg_t temp;

    temp.pid = getpid();

    strncpy(temp.user, user, MAX_USER-1); temp.user[MAX_USER-1] = '\0';

    strncpy(temp.text, teks, MAX_TEXT-1); temp.text[MAX_TEXT-1] = '\0';

    temp.ts = time(NULL);

    sem_wait(mutex);

    int pos;

    if (shm->count < MAX_MSG) {

        pos = (shm->head + shm->count) % MAX_MSG;

        shm->msgs[pos] = temp;

        shm->count++;

    } else {

        pos = shm->head;

        shm->msgs[pos] = temp;

        shm->head = (shm->head + 1) % MAX_MSG;

    }

    sem_post(mutex);

}

```

```

int main(void) {

```

```
char username[MAX_USER];

printf("Masukkan nama pengguna: ");

if (!fgets(username, sizeof(username), stdin)) return 1;

username[strcspn(username, "\n")] = '\0';

if (strlen(username) == 0) strncpy(username, "Anonim", sizeof(username));


int baru = 0;

shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_EXCL, 0660);

if (shm_fd >= 0) {

    baru = 1;

    if (ftruncate(shm_fd, sizeof(chat_shm_t)) == -1) {

        perror("ftruncate");

        close(shm_fd);

        shm_unlink(SHM_NAME);

        return 1;

    }

} else if (errno == EEXIST) {

    shm_fd = shm_open(SHM_NAME, O_RDWR, 0);

    if (shm_fd == -1) {

        perror("shm_open");

        return 1;

    }

} else {

    perror("shm_open");

    return 1;

}
```

```

    shm = mmap(NULL, sizeof(chat_shm_t), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);

    if (shm == MAP_FAILED) {

        perror("mmap");

        close(shm_fd);

        if (baru) shm_unlink(SHM_NAME);

        return 1;

    }

    mutex = sem_open(SEM_NAME, O_CREAT, 0660, 1);

    if (mutex == SEM_FAILED) {

        perror("sem_open");

        munmap(shm, sizeof(chat_shm_t));

        close(shm_fd);

        if (baru) shm_unlink(SHM_NAME);

        return 1;

    }

    sem_wait(mutex);

    if (baru) {

        shm->clients = 0;

        shm->head = 0;

        shm->count = 0;

    }

    shm->clients++;

    int jumlah = shm->clients;

    sem_post(mutex);

    atexit(cleanup_on_exit);

    printf("\n=== CHAT IPC ===\n");

```

```
printf("Pengguna: %s | PID: %d | Klien terhubung: %d\n", username, (int)getpid(), jumlah);  
printf("Perintah: kirim <pesan> | lihat | keluar\n\n");
```

```
char line[512];
```

```
while (1) {
```

```
    printf("> ");
```

```
    if (!fgets(line, sizeof(line), stdin)) break;
```

```
    line[strcspn(line, "\n")] = '\0';
```

```
    if (strlen(line) == 0) continue;
```

```
    if (strncmp(line, "kirim ", 6) == 0) {
```

```
        const char *msg = line + 6;
```

```
        if (strlen(msg) == 0) {
```

```
            printf("Pesan kosong.\n");
```

```
        } else {
```

```
            kirim_pesan(username, msg);
```

```
        }
```

```
    } else if (strcmp(line, "lihat") == 0) {
```

```
        tampilkan_riwayat();
```

```
    } else if (strcmp(line, "keluar") == 0) {
```

```
        break;
```

```
    } else {
```

```
        printf("Perintah tidak dikenal. Gunakan: kirim <pesan> | lihat | keluar\n");
```

```
    }
```

```
}
```

```
printf("Keluar...\n");
```

```
return 0;
```

```
}
```

Output

```
yos@yos-QEMU-Virtual-Machine:~$ ./chat_ipc
Masukkan nama pengguna: Yos

=== CHAT IPC ===
Pengguna: Yos | PID: 4368 | Klien terhubung: 2
Perintah: kirim <pesan> | lihat | keluar

> kirim halo semua!
> kirim bagaimana kabarmu?
> keluar
Keluar...
yos@yos-QEMU-Virtual-Machine:~$ ./chat_ipc
Masukkan nama pengguna: Mex

=== CHAT IPC ===
Pengguna: Mex | PID: 4378 | Klien terhubung: 2
Perintah: kirim <pesan> | lihat | keluar

> kirim hai user1!
> kirim baik, terima kasih
> lihat
=== CHAT ROOM ===
Yos> halo semua!
Yos> bagaimana kabarmu?
Mex> hai user1!
Mex> baik, terima kasih
> 
```

Analisis Hasil

- Program berhasil mengimplementasikan **komunikasi antar proses** menggunakan **Message Queue**.
- Proses anak menerima pesan, sedangkan proses induk mengirim pesan.
- Pesan dapat ditampilkan secara real-time di terminal lain.
- Jika salah satu user mengetik **exit**, proses keluar dari chat.

Kesimpulan

1. IPC memungkinkan proses saling bertukar data secara efisien.
2. **Message Queue** adalah metode IPC yang cocok untuk aplikasi chat karena dapat menyimpan pesan sementara dan digunakan antar proses yang berbeda.
3. Program berhasil menunjukkan implementasi chat sederhana berbasis IPC di Linux.